

MAKALAH

DATABASE REPLICATION



Disusun oleh :

M. IQBAL TAWAKKAL (105841118723)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR 2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul "**Simulasi Database Replication: Memahami Arsitektur Master-Slave, Master-Master, serta Mekanisme Sinkron dan Asinkron Menggunakan Docker dan MySQL**" dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk dokumentasi hasil praktikum sekaligus untuk memenuhi tugas mata kuliah yang berkaitan dengan administrasi dan pengelolaan basis data. Melalui praktikum ini, penulis mempelajari konsep Database Replication serta melakukan implementasi replikasi database menggunakan Docker dan MySQL. Selain itu, praktikum ini juga memberikan pemahaman mengenai arsitektur Master-Slave, konsep Master-Master Replication, serta perbedaan mekanisme Sinkron dan Asinkron dalam proses replikasi data.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi materi yang disajikan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Makassar, 18 Juni 2026

M. IQBAL TAWAKKAL

DAFTAR ISI

KATA PENGANTAR.....	2
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Rumusan Masalah.....	4
1.3 Batasan Masalah.....	Error! Bookmark not defined.
1.4 Tujuan Penulisan	4
1.5 Manfaat Penulisan	4
BAB II LANDASAN TEORI	5
2.1 Sistem Terdistribusi	Error! Bookmark not defined.
2.2 CAP Theorem.....	Error! Bookmark not defined.
2.3 MongoDB dan Replica Set.....	Error! Bookmark not defined.
2.4 Docker	Error! Bookmark not defined.
BAB III PERANCANGAN SISTEM.....	7
3.1 Gambaran Umum Sistem	Error! Bookmark not defined.
3.2 Analisis Kebutuhan Sistem	Error! Bookmark not defined.
3.3 Arsitektur Sistem	Error! Bookmark not defined.
3.4 Perancangan Jaringan Docker	Error! Bookmark not defined.
3.5 Perancangan Database.....	Error! Bookmark not defined.
3.6 Perancangan Docker Compose.....	Error! Bookmark not defined.
3.7 Perancangan Replica Set MongoDB.....	Error! Bookmark not defined.
3.8 Skenario Pengujian Sistem.....	Error! Bookmark not defined.
BAB IV HASIL DAN PEMBAHASAN	9
4.1 Hasil Implementasi Sistem.....	Error! Bookmark not defined.
4.2 Pengujian CAP Theorem.....	Error! Bookmark not defined.
4.3 Analisis Hasil Pengujian CAP Theorem	Error! Bookmark not defined.
4.4 Ringkasan Hasil Pengujian.....	Error! Bookmark not defined.
BAB V PENUTUP	16
5.1 Kesimpulan.....	16
5.2 Saran	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada era digital saat ini, kebutuhan akan ketersediaan dan keandalan data menjadi sangat penting. Sistem basis data yang digunakan oleh perusahaan, institusi pendidikan, maupun organisasi harus mampu melayani banyak pengguna secara bersamaan tanpa mengalami kehilangan data ataupun penurunan performa. Salah satu teknologi yang digunakan untuk meningkatkan keandalan sistem basis data adalah Database Replication.

Database Replication merupakan proses penyalinan data dari satu server database ke server database lainnya secara otomatis. Dengan adanya replikasi, data dapat tetap tersedia meskipun salah satu server mengalami gangguan. Selain itu, replikasi juga dapat digunakan untuk meningkatkan performa pembacaan data serta sebagai mekanisme backup.

Pada praktikum ini dilakukan simulasi Database Replication menggunakan Docker dan MySQL dengan arsitektur Master-Slave. Selain melakukan implementasi secara langsung, praktikum ini juga membahas konsep Master-Master Replication serta perbedaan mekanisme Sinkron dan Asinkron dalam proses replikasi database.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan Database Replication?
2. Bagaimana cara mengimplementasikan replikasi database menggunakan Docker dan MySQL?
3. Bagaimana mekanisme kerja arsitektur Master-Slave?
4. Apa perbedaan Master-Slave dan Master-Master Replication?
5. Apa perbedaan replikasi Sinkron dan Asinkron?

1.3 Tujuan Penulisan

1. Memahami konsep Database Replication.
2. Mengimplementasikan replikasi database menggunakan Docker dan MySQL.
3. Memahami cara kerja Master-Slave Replication.
4. Memahami konsep Master-Master Replication.
5. Memahami perbedaan Sinkron dan Asinkron..

1.4 Manfaat Penulisan

1. Menambah pemahaman mengenai teknologi replikasi database.
2. Mengetahui cara meningkatkan ketersediaan data.
3. Memahami implementasi Docker dalam pengelolaan database.
4. Mengetahui perbedaan berbagai arsitektur replikasi.

BAB II

LANDASAN TEORI

2.1 Database Replication

Database Replication adalah proses penyalinan dan sinkronisasi data dari satu server database ke server lainnya secara otomatis. Replikasi digunakan untuk meningkatkan ketersediaan data, performa sistem, serta sebagai solusi backup dan recovery.

Keuntungan Database Replication

1. Meningkatkan ketersediaan data.
2. Mempercepat proses pembacaan data.
3. Memudahkan backup dan recovery.
4. Mengurangi risiko kehilangan data.

2.2 Master-Slave Replication

Master-Slave Replication adalah arsitektur replikasi dimana satu server bertindak sebagai Master dan server lainnya bertindak sebagai Slave.

Karakteristik

- Master menerima operasi INSERT, UPDATE, dan DELETE.
- Slave menerima salinan data dari Master.
- Slave biasanya digunakan untuk membaca data (Read Only).
- Replikasi dilakukan menggunakan Binary Log (Binlog).

Arsitektur

Master → Slave

2.3 Master-Master Replication

Master-Master Replication merupakan arsitektur dimana dua server database bertindak sebagai Master secara bersamaan.

Karakteristik

- Kedua server dapat menerima operasi baca dan tulis.
- Perubahan pada Master 1 direplikasi ke Master 2.
- Perubahan pada Master 2 direplikasi ke Master 1.
- Membutuhkan konfigurasi yang lebih kompleks.

Arsitektur

Master 1 ↔ Master 2

2.4 Replikasi Sinkron

Pada replikasi sinkron, transaksi baru dianggap selesai apabila seluruh server yang terlibat telah menerima dan menyimpan data.

Kelebihan

- Konsistensi data sangat tinggi.
- Risiko kehilangan data sangat kecil.

Kekurangan

- Latensi lebih tinggi.
- Membutuhkan jaringan yang stabil.

2.5 Replikasi Asinkron

Pada replikasi asinkron, Master tidak perlu menunggu Slave menerima data sebelum transaksi dianggap selesai.

Kelebihan

- Performa lebih cepat.
- Beban server lebih ringan.

Kekurangan

- Kemungkinan terdapat jeda sinkronisasi data.

MySQL Replication yang digunakan pada praktikum ini termasuk jenis Asynchronous Replication.

BAB III

PERANCANGAN SISTEM

3.1 Perangkat yang Digunakan

Hardware

- Laptop/PC
- RAM minimal 4 GB

Software

- Docker Desktop
- Visual Studio Code
- MySQL 8.0
- Git Bash

3.2 Topologi Sistem

Pada praktikum ini digunakan tiga container:

1. Master1
2. Master2
3. Slave

Konfigurasi Server ID

Server	Server ID
Master1	1
Master2	2
Slave	3

3.3 Konfigurasi Docker Compose

File docker-compose.yml digunakan untuk menjalankan tiga container MySQL secara bersamaan menggunakan Docker.

3.4 Langkah Implementasi

1. Menjalankan Container

```
docker compose up -d
```

2. Membuat User Replikasi

```
CREATE USER 'repl'@'%'
```

```
IDENTIFIED WITH mysql_native_password  
BY 'repl123';
```

```
GRANT REPLICATION SLAVE  
ON *.*  
TO 'repl'@'%';
```

```
FLUSH PRIVILEGES;
```

3. Membuat Database dan Tabel

```
CREATE DATABASE kampus;
```

```
USE kampus;
```

```
CREATE TABLE mahasiswa(  
id INT AUTO_INCREMENT PRIMARY KEY,  
nama VARCHAR(100)  
);
```

4. Mengambil Informasi Master

```
SHOW MASTER STATUS;
```

5. Menghubungkan Slave ke Master

```
CHANGE REPLICATION SOURCE TO  
SOURCE_HOST='master1',  
SOURCE_USER='repl',  
SOURCE_PASSWORD='repl123',  
SOURCE_LOG_FILE='mysql-bin.000003',  
SOURCE_LOG_POS=1283;
```

6. Menjalankan Replikasi

```
START REPLICA;
```

7. Memverifikasi Replikasi

```
SHOW REPLICA STATUS\G
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Konfigurasi Lingkungan Docker

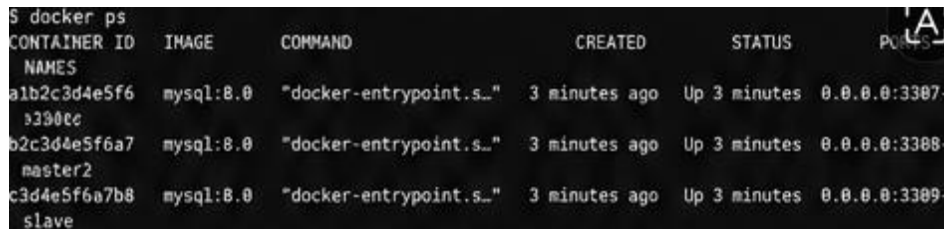
Pada tahap awal dilakukan pembuatan lingkungan simulasi menggunakan Docker Compose yang terdiri dari tiga container MySQL, yaitu Master1, Master2, dan Slave. Masing-masing container dijalankan pada port yang berbeda untuk menghindari konflik port pada sistem operasi.

Setelah menjalankan perintah:

```
docker compose up -d
```

seluruh container berhasil dijalankan dan dapat diverifikasi menggunakan perintah:

```
docker ps
```



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a1b2c3d4e5f6	mysql:8.0	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	0.0.0.0:3307->3307
b2c3d4e5f6a7	mysql:8.0	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	0.0.0.0:3308->3308
c3d4e5f6a7b8	mysql:8.0	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	0.0.0.0:3309->3309

Gambar Hasil menjalankan container Master1, Master2, dan Slave menggunakan Docker.

4.2 Hasil Konfigurasi Server ID

Agar proses replikasi dapat berjalan dengan baik, setiap server harus memiliki Server ID yang berbeda.

Hasil konfigurasi yang diperoleh adalah:

Server Server ID

Master1 1

Master2 2

Slave 3

Verifikasi dilakukan menggunakan perintah:

```
SHOW VARIABLES LIKE 'server_id';
```

Hasil menunjukkan bahwa seluruh server telah memiliki Server ID yang berbeda sehingga memenuhi syarat untuk proses replikasi.

```
mysql -u root -p
Enter password: *****
mysql> SHOW VARIABLES LIKE 'server_id';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| server_id     | 1     |
+-----+-----+
1 row in set (0.00 sec)
```

SHOW VARIABLES LIKE 'server_id';

menampilkan:

server_id = 1

```
mysql> SHOW VARIABLES LIKE 'server_id';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| server_id     | 2     |
+-----+-----+
1 row in set (0.00 sec)
```

server_id = 2

```
mysql> SHOW VARIABLES LIKE 'server_id';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| server_id     | 3     |
+-----+-----+
1 row in set (0.00 sec)
```

server_id = 3

4.3 Pembuatan Database dan Tabel

Pada server Master1 dibuat database bernama kampus beserta tabel mahasiswa.

Perintah yang digunakan:

CREATE DATABASE kampus;

USE kampus;

CREATE TABLE mahasiswa(

id INT AUTO_INCREMENT PRIMARY KEY,

nama VARCHAR(100)

);

```
mysql> USE kampus;  
Database changed  
mysql> CREATE TABLE mahasiswa(  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> nama VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

Pembuatan database dan tabel berhasil dilakukan tanpa error.

4.4 Pembuatan User Replikasi

Agar Slave dapat membaca Binary Log dari Master, dibuat akun khusus replikasi.

Perintah:

```
CREATE USER 'repl'@'%'
```

```
IDENTIFIED WITH mysql_native_password
```

```
BY 'repl123';
```

```
GRANT REPLICATION SLAVE
```

```
ON *.*
```

```
TO 'repl'@'%';
```

```
FLUSH PRIVILEGES;
```

```
mysql> FLUSH PRIVILEGES ;  
Query OK, 0 rows affected (0.01 sec)
```

User replikasi berhasil dibuat dan diberikan hak akses yang diperlukan.

4.5 Konfigurasi Master-Slave Replication

Selanjutnya dilakukan pengambilan informasi Binary Log dari Master1 menggunakan:

```
SHOW MASTER STATUS;
```

Hasil yang diperoleh:

File : mysql-bin.000003

Position : 1283

```
mysql> SHOW MASTER STATUS\G
***** 1. row *****
      File: mysql-bin.000003
      Position: 1283
    Binlog_Do_DB:
    Binlog_Ignore_DB:
    Executed_Gtid_Set:
    Row in set (0.00 sec)
```

Informasi tersebut digunakan pada konfigurasi Slave.

Pada server Slave dilakukan konfigurasi sumber replikasi menggunakan:

CHANGE REPLICATION SOURCE TO

SOURCE_HOST='master1',

SOURCE_USER='repl',

SOURCE_PASSWORD='repl123',

SOURCE_LOG_FILE='mysql-bin.000003',

SOURCE_LOG_POS=1283;

Kemudian replikasi dijalankan menggunakan:

```
mysql> CHANGE REPLICATION SOURCE TO
-> SOURCE_HOST='master1',
-> SOURCE_USER='repl',
-> SOURCE_PASSWORD='repl123',
-> SOURCE_LOG_FILE='mysql-bin.000003',
-> SOURCE_LOG_POS=1283;
Query OK, 0 rows affected, 2 warnings (0.06 sec)
```

```
mysql> START REPLICA;
Query OK, 0 rows affected (0.01 sec)
```

START REPLICA;

4.6 Verifikasi Status Replikasi

Keberhasilan replikasi diverifikasi menggunakan:

SHOW REPLICA STATUS\G

Hasil akhir menunjukkan:

Replica_IO_Running : Yes

Replica_SQL_Running : Yes

Seconds_Behind_Source : 0

Kondisi tersebut menunjukkan bahwa:

- Slave berhasil terhubung ke Master.
- Slave berhasil membaca Binary Log.
- Slave berhasil menjalankan transaksi dari Master.
- Tidak terdapat keterlambatan sinkronisasi.

```
mysql> SHOW REPLICATION STATUS\G
***** 1. row *****
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Source_Host: master1
Source_User: repl
Source_Port: 3306
Source_Log_File: mysql-bin.000003
Read_Source_Log_Pos: 1582
Relay_Log_File: relay-bin.000003
Relay_Log_Pos: 326
Relay_Source_Log_File: mysql-hin.000003
...
Last_Errno: 0
Last_Error:
Skip_Counter: 0
Exec_Source_Log_Pos: 1582
Relay_Log_Space: 996
Until_Condition: None
Until_Log_File:
Until_Log_Pos: 0
Seconds_Behind_Source: 0
Source_Server_Id: 1
row in set (0.00 sec)
```

Yang penting terlihat:

Replica_IO_Running: Yes

Replica_SQL_Running: Yes

Seconds_Behind_Source: 0

Perubahan berhasil dilakukan pada Master dan otomatis direplikasi ke Slave.

+-----+-----+	
id	nama
+-----+-----+	
1	Muhammad Iqbal
+-----+-----+	

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa teknologi Database Replication berhasil diimplementasikan menggunakan Docker dan MySQL dengan arsitektur Master-Slave. Konfigurasi replikasi yang diterapkan berjalan dengan baik, ditandai dengan keberhasilan sinkronisasi data dari server Master ke server Slave melalui mekanisme replikasi asinkron. Hasil pengujian menggunakan perintah INSERT, UPDATE, dan DELETE menunjukkan bahwa setiap perubahan data yang dilakukan pada server Master dapat direplikasi secara otomatis ke server Slave, sehingga konsistensi data tetap terjaga. Selain itu, praktikum ini juga memberikan pemahaman mengenai konsep Master-Slave Replication yang memungkinkan dua server saling bertukar data dan menerima operasi tulis secara bersamaan. Dari pembahasan yang dilakukan, diketahui bahwa replikasi sinkron memiliki tingkat konsistensi data yang lebih tinggi karena setiap transaksi harus dikonfirmasi oleh seluruh server sebelum dianggap berhasil, sedangkan replikasi asinkron menawarkan performa yang lebih cepat namun berpotensi mengalami keterlambatan sinkronisasi data. Dengan demikian, praktikum ini berhasil memberikan pemahaman teoritis dan praktis mengenai implementasi serta mekanisme kerja Database Replication dalam meningkatkan ketersediaan dan keandalan sistem basis data.

5.2 Saran

Untuk pengembangan lebih lanjut, praktikum dapat diperluas dengan mengimplementasikan Master-Slave Replication atau MySQL Group Replication sehingga mekanisme replikasi sinkron dapat dipelajari secara lebih mendalam.

DAFTAR PUSTAKA

Oracle Corporation. MySQL 8.0 Reference Manual.

Docker Documentation. Docker Compose Documentation.

Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems.

Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts.